- **This guide outlines changes to make OpenLUR more adaptable to different environments**

Making Database Connection Parameters Configurable

Why this change?

Currently, the OpenLUR codebase has hardcoded database connection parameters (`172.18.0.2` as host and 5432 as port), which causes connection issues across different systems. So make these parameters configurable through command-line arguments.

Files to modify:

1. database_generation_utils.py

- Added *"db_host"* and *"db_port"* parameters to all database connection functions
- Updated all psycopg2 connection calls to use these parameters
- Added the parameters to the command-line arguments for the script when used directly

2. OSMRequestor.py

- Modified the **`Requestor`** class constructor to accept *"db_host"* and *"db_port"* parameters
- Used these parameters when establishing the database connection

3. FeatureGenerator.py

- Added *"db_host"* and *"db_port* parameters to the **`FeatureGenerator`** constructor
- Stored these parameters as class attributes
- Passed them to the **`OSMRequestor.Requestor`** when creating features
- Added proper connection closing after feature extraction
- Updated the main function to accept and forward these parameters

4. osm_feature_generation.py

- Added *"--db-host"* and *"--db-port"* command-line arguments
- Modified the processing functions to use these parameters when calling **`create_db`** and initializing **`FeatureGenerator`**

How to Use

With these changes, you can now run OpenLUR and specify custom database connection parameters:

```
# For map-based feature generation:
python3 osm_feature_generation.py map bremen_db 53.02 53.20 8.56 8.96 -p 16 --db-host 127.0.0.1 --db-port 5432

# For file-based feature generation:
python3 osm_feature_generation.py file bremen_db input_locations.csv -p 16 --db-host 127.0.0.1 --db-port 5432
```

NOTE : The default values remain *172.18.0.2* for host and *5432* for port, but they can now be easily customized to match your specific environment.

How I test

I can't directly test it on other machines, I need to verify that the modifications work correctly by simulating different database connection scenarios on my own machine. Here's how i do that:

1. Test with Incorrect Connection Parameters

The simplest verification is to intentionally use incorrect connection parameters and confirm that the script handles this appropriately:

```
python3 osm_feature_generation.py map bremen_db 53.02 53.20 8.56 8.96 -p 16 --db-host 192.168.0.123
--db-port 5555
```

This should fail with a connection error, proving that the script is actually using given parameters instead of the hardcoded ones.

2. Test with Docker Container IP Discovery

If using Docker, determine container's current IP and use that explicitly:

```
# Find your PostgreSQL container ID or Name
docker ps

# Get the container's IP address
DB_IP=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
<your_container_id or name>)

# Run with the discovered IP
python3 osm_feature_generation.py map bremen_db 53.02 53.20 8.56 8.96 -p 16 --db-host $DB_IP --db-
port 5432
```

If the script works with the explicitly provided IP, it confirms the changes are working correctly.

3. Add Debug Logging

Add some temporary debug statements to verify connection parameters are correctly passed:

```
# In create_db function in database_generation_utils.
    print(f"DEBUG: Attempting to connect to database at {db_host}:{db_port}")
```

These debug prints will help confirm your parameters are correctly propagated through the code.